

PRÁCTICA 05

Arquitectura en N Capas con C# y SQL Server

UNIDAD DE APRENDIZAJE 2

EDICIÓN REVISADA — VS 2022 / VS 2026

- Migración desde ADO.NET monolítico (Práctica 04) a 3 capas
- Construcción de proyectos: BE, ADO, BL y GUI en Visual Studio
- Implementación del CRUD completo con procedimientos almacenados
- Validaciones en la capa de negocio y diseño de la presentación
- Capítulo de errores comunes y cómo resolverlos

/ ÍNDICE

- 1. Introducción y antes de empezar
- 2. ¿Qué es el patrón N Capas?
- 3. Base de datos VentasLeon (SQL Server)
- 4. Creación de la solución y proyectos
- 5. Paquetes NuGet en la capa ADO
- 6. Referencias entre proyectos
- 7. Configuración App.config
- 8. Capa de Entidades (BE)
- 9. Capa de Acceso a Datos (ADO)
- 10. Capa de Negocio (BL)
- 11. Capa de Presentación (GUI)
- 12. Ejecución y pruebas
- 13. Errores comunes y soluciones
- 14. Conclusiones

/ 1. ANTES DE EMPEZAR — LEE ESTO

Esta es la **edición revisada** del manual de la Práctica 05, basada en una sesión real de prueba donde se identificaron los puntos donde los alumnos se traban. Léelo de corrido la primera vez sin escribir nada.

Prerequisitos

- Tener completada la **Práctica 04** (ADO.NET monolítico).
- Visual Studio 2022 o 2026 instalado con la carga de trabajo **.NET desktop development**.
- SQL Server (Express o Developer) y SQL Server Management Studio (SSMS).
- Tu cadena de conexión de la Práctica 04 (la usaremos como referencia).

Convenciones de este manual

Símbolo	Significado
■ NOTA	Información complementaria útil.
■ RECUERDA	Tip o buena práctica que vale la pena recordar.
■ CUIDADO	Punto delicado — si lo haces mal, te toca deshacer.
■ CHECKPOINT	Verifica que esto funciona antes de continuar.
■ NOTA VS	Diferencias entre Visual Studio 2022 y 2026.

Estrategia recomendada: ir paso a paso con checkpoints

En vez de pegar todo el código y al final compilar, este manual te pide **compilar después de cada capa**. Si la BE no compila, no sirve seguir con ADO. Detectar errores temprano te ahorra horas.

■ **RECUERDA:** Tiempo estimado: **2 a 3 horas** la primera vez. Si te traba un error, ve al **capítulo 13 (Errores comunes)** antes de desesperarte.

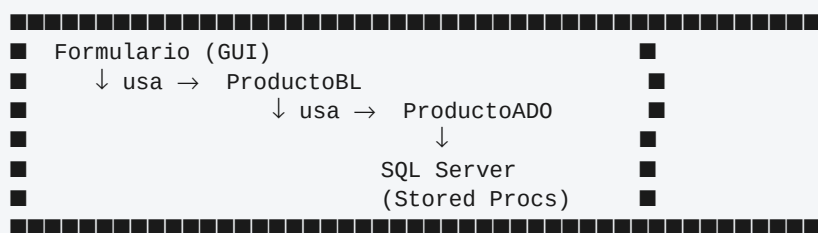
/ 2. ¿QUÉ ES EL PATRÓN N CAPAS?

Es un patrón arquitectónico que consiste en **agrupar en una solución un conjunto de proyectos independientes pero referenciados entre sí**, donde cada uno tiene una responsabilidad clara.

Las 4 partes de la solución

CAPA	RESPONSABILIDAD
1. Entidades (BE)	Clases POCO que representan las tablas. Solo propiedades, sin lógica. Viajan entre todas las capas.
2. Acceso a Datos (ADO)	Librería que invoca a SQL Server con ADO.NET. Aquí van SqlConnection, SqlCommand, SqlDataAdapter.
3. Negocio (BL)	Capa intermedia con validaciones y reglas. Recibe llamadas de la GUI y delega en la ADO.
4. Presentación (GUI)	Windows Forms. Contiene los formularios y la lógica de UI. Solo conoce a la capa BL.

Flujo de la información



La capa GUI NUNCA conoce a la capa ADO directamente.
 La capa ADO NUNCA conoce a la capa GUI.
 Todo pasa por la capa BL.

Ventajas

- **Mantenimiento:** los cambios se hacen en la capa correspondiente.
- **Reutilización:** la misma BL/ADO sirve a varios formularios.
- **Depuración:** los errores se localizan rápido.
- **Trabajo en equipo:** distintas personas pueden trabajar en distintas capas.
- **Cambio de tecnología:** migrar de SQL Server a Oracle solo afecta a la ADO.

/ 3. BASE DE DATOS: VentasLeon

Antes de tocar Visual Studio, vamos a crear la base de datos completa. **Este orden importa:** si intentas ejecutar la aplicación sin BD, fallará con error de conexión.

■ **CUIDADO: NO avances al capítulo 4 hasta confirmar que el script SQL ejecutó correctamente y la consulta de prueba devuelve 5 productos.**

PASO 1: Abrir SSMS y conectarse al servidor

- Abre **SQL Server Management Studio**.
- Server name: `.\SQLEXPRESS` (o el que usaste en Practica04).
- Authentication: **Windows Authentication**.
- Clic en **Connect**.

PASO 2: Abrir una nueva ventana de consulta

- Una vez conectado, clic en **New Query** (o Ctrl+N).
- Se abrirá una pestaña en blanco para SQL.

PASO 3: Pegar y ejecutar el script completo

El script completo está en el archivo `01_CrearBD_VentasLeon.sql` que viene en el paquete. Cópialo, pégalo en SSMS y pulsa **F5**.

Contenido del script (para referencia)

Crea la base de datos, las tablas, la vista y los 7 procedimientos almacenados:

```
-- 1. Crea la BD
CREATE DATABASE VentasLeon;
GO
USE VentasLeon;
GO

-- 2. Tablas: Tb_Categoria, Tb_UnidadMedida, Tb_Producto
-- 3. Datos iniciales: 5 categorías, 5 UMs, 5 productos
-- 4. Vista: VW_VistaProductos (JOIN con descripciones)
-- 5. Procedimientos:
--     usp_ListarProducto
--     usp_ConsultarProducto
--     usp_InsertarProducto    (autogenera P001, P002...)
--     usp_ActualizarProducto
--     usp_EliminarProducto
--     usp_ListarCategoria    (para combos)
--     usp_ListarUM           (para combos)
```

■ **CHECKPOINT:** Después de ejecutar el script, el mensaje de salida debe decir:

Commands completed successfully

Y en la pestaña **Results** debes ver los 5 productos de muestra.

PASO 4: Verificación obligatoria

Ejecuta esta consulta de prueba:

```
EXEC usp_ListarProducto;
```

Debe devolver **5 filas** con los productos: Arroz, Aceite, Inca Kola, Detergente, Leche.

■ **CUIDADO:** Si la consulta no devuelve 5 filas, hay un problema. Revisa el panel **Messages** de SSMS para ver el error y corrige antes de avanzar.

/ 4. CREACIÓN DE LA SOLUCIÓN

PASO 1: Crear la solución en blanco

- Abre **Visual Studio** → **Crear un proyecto nuevo**.
- Buscador: escribe “**Solución en blanco**”.
- Selecciónala → **Siguiente**.
- Nombre de la solución: **Practica05_Capas**
- Ubicación: una carpeta de tu preferencia.
- Clic en **Crear**.

■ **NOTA:** Una solución en blanco es solo un contenedor (archivo `.sln`). Dentro de ella agregaremos los 4 proyectos.

¿Dónde queda el Explorador de soluciones?

Es el panel del lado **derecho** de Visual Studio. Ahí verás una línea que dice Solución "Practica05_Capas" (0 proyectos). Sobre esa línea harás clic derecho para agregar cada proyecto.

■ **RECUERDA:** Si por error cierras el Explorador de soluciones, recupéralo con el menú **Ver** → **Explorador de soluciones** (atajo `Ctrl + Alt + L`).

PASO 2: Agregar la capa BE — ¡CUIDADO con la plantilla!

■ **CUIDADO:** Hay 8 plantillas distintas que contienen las palabras "Biblioteca de clases". Si eliges la equivocada, todo el patrón se rompe y tendrás que eliminar el proyecto y empezar de nuevo.

Pasos:

- Clic derecho sobre la solución → **Agregar** → **Nuevo proyecto.**
- Buscador: **“biblioteca de clases”**.
- Verás varios resultados. Haz **scroll hasta el final.**

■ La plantilla correcta

```
■ Biblioteca de clases ■
■ Proyecto para crear una biblioteca de clases para .NET ■
■ o .NET Standard ■
■ ■
■ Tags: [C#] [Android] [Linux] [macOS] [Windows] ■
```

↑ Esta es la que necesitas. Identifícala por los tags: debe decir Android, Linux, macOS y Windows juntos. Es la versión multiplataforma más limpia.

■ NO elijas estas (son trampas)

- ■ Biblioteca de clases de **WPF**
- ■ Biblioteca de clases de **Windows Forms**
- ■ Biblioteca de **control personalizada** (de WPF o WinForms)
- ■ Biblioteca de **controles** de Windows Forms
- ■ Cualquiera que diga **(.NET Framework)** al final

Una vez seleccionada la correcta:

- Clic en **Siguiente**.
- Nombre del proyecto: **ProyVentas_BE**
- Clic en **Siguiente**.
- Framework: **.NET 10.0** (o el más alto disponible).
- Clic en **Crear**.

■ **NOTA:** Si Visual Studio crea automáticamente un archivo `Class1.cs`, elimínalo (clic derecho → Eliminar). Si no aparece, mejor — depende de la versión.

PASO 3: Agregar las otras 2 bibliotecas (ADO y BL)

Repite exactamente el proceso anterior dos veces más, eligiendo la **misma plantilla** (Biblioteca de clases multiplataforma):

- Proyecto 2: **ProyVentas_ADO**
- Proyecto 3: **ProyVentas_BL**

PASO 4: Agregar la capa GUI — plantilla diferente

■ **CUIDADO:** Esta plantilla NO es "Biblioteca de clases", es "**Aplicación de Windows Forms**". Es la única que tiene formulario.

- Clic derecho sobre la solución → **Agregar** → **Nuevo proyecto**.
- Buscador: "**windows forms**".
- Selecciona **Aplicación de Windows Forms (C#)**.
- ¡**OJO!** Asegúrate de que **no** diga "(.NET Framework)" al final.
- Clic en **Siguiente**.
- Nombre: **ProyVentas_GUI**
- Framework: **.NET 10.0** → **Crear**.

PASO 5: Establecer ProyVentas_GUI como proyecto de inicio

- Clic derecho sobre ProyVentas_GUI en el Explorador.
- Selecciona **Establecer como proyecto de inicio**.
- Verifica que el nombre aparezca en **negrita**.

■ **CHECKPOINT:** El Explorador de soluciones debe verse así:

Solución 'Practica05_Capas' (4 de 4 proyectos)

```

■■■ ProyVentas_ADO
■■■ ProyVentas_BE
■■■ ProyVentas_BL
■■■ ProyVentas_GUI ← en negrita (proyecto de inicio)
  
```

Los proyectos aparecen en orden alfabético, no importa el orden en que los hayas creado.

/ 5. PAQUETES NuGet EN LA CAPA ADO

La capa de datos necesita dos paquetes para hablar con SQL Server y leer el archivo de configuración.

■ **CUIDADO:** Instala estos paquetes **SOLAMENTE** en **ProyVentas_ADO**, no en los otros proyectos. La ADO es la única que habla con SQL Server.

PASO 1: Abrir el administrador de NuGet

- Clic derecho sobre **ProyVentas_ADO**.
- Selecciona **Administrar paquetes NuGet...**
- Se abre una nueva pestaña.

■ **NOTA DE VERSIÓN VS:** En **VS 2026** a veces se abre la vista “Administrar paquetes para la solución” en lugar de la vista por proyecto. Esa vista muestra una lista con checks de los 4 proyectos. Si te aparece esa, simplemente **marca solo el check de ProyVentas_ADO** y desmarca los demás. El resultado es el mismo.

PASO 2: Instalar Microsoft.Data.SqlClient

- Pestaña **Examinar** (arriba a la izquierda).
- Buscador: `Microsoft.Data.SqlClient`
- Selecciona el **primer resultado** (autor: *Microsoft*).

■ **CUIDADO:** Hay paquetes con nombres muy parecidos. **NO instales:**

- `Microsoft.Data.SqlClient.SNI` (es interno)
- `System.Data.SqlClient` (dice DEPRECATED, está obsoleto)
- Los que dicen `runtime.native...` (son internos)

El correcto es solo `Microsoft.Data.SqlClient`, sin sufijos.

- Clic en **Instalar** (panel derecho).
- Aceptar términos de licencia.
- Espera a que termine (verás actividad en la ventana **Salida**).

PASO 3: Instalar System.Configuration.ConfigurationManager

- En el mismo buscador, escribe: `System.Configuration.ConfigurationManager`
- Primer resultado (autor: *Microsoft*) → **Instalar** → **Acepto**.

■ **CHECKPOINT:** Verifica en el Explorador de soluciones que en **ProyVentas_ADO** aparezcan:

```
ProyVentas_ADO
  ■■■ Dependencias
    ■■■ Paquetes
      ■■■ Microsoft.Data.SqlClient (7.0.x)      ■
      ■■■ System.Configuration.ConfigurationManager  ■
```

/ 6. REFERENCIAS ENTRE PROYECTOS

Las dependencias entre capas deben ser **estrictamente unidireccionales**:

```
GUI ■referencia→ BL (y BE)
BL ■referencia→ ADO (y BE)
ADO ■referencia→ BE
BE ■referencia→ (nadie, es el núcleo)
```

■ **CUIDADO:** La capa GUI NO debe referenciar a ADO. Si lo haces, rompes el patrón. Toda comunicación con la base de datos debe pasar por la capa BL.

PASO 1: Referencia BE → ProyVentas_ADO

- Dentro de ProyVentas_ADO, clic derecho sobre **Dependencias**.
- Selecciona **Agregar referencia de proyecto...**
- En la ventana, izquierda haz clic en **Proyectos**.
- Marca el check de **ProyVentas_BE**.
- Clic en **Aceptar**.

PASO 2: Referencias ADO + BE → ProyVentas_BL

- Dentro de ProyVentas_BL, clic derecho sobre **Dependencias**.
- **Agregar referencia de proyecto...**
- Marca los dos checks: **ProyVentas_ADO** y **ProyVentas_BE**.
- Clic en **Aceptar**.

PASO 3: Referencias BL + BE → ProyVentas_GUI

- Dentro de ProyVentas_GUI, clic derecho sobre **Dependencias**.
- **Agregar referencia de proyecto...**
- Marca los dos checks: **ProyVentas_BL** y **ProyVentas_BE**.
- **NO marques ProyVentas_ADO.**
- Clic en **Aceptar**.

■ **CHECKPOINT:** Resultado esperado en el Explorador de soluciones:

```
ProyVentas_ADO/Dependencias/
  ■■■ Paquetes (Microsoft.Data.SqlClient, ConfigurationManager)
  ■■■ Proyectos
    ■■■ ProyVentas_BE ■

ProyVentas_BE/Dependencias/
  ■■■ (solo Analizadores y Marcos) ■

ProyVentas_BL/Dependencias/
  ■■■ Proyectos
    ■■■ ProyVentas_ADO ■
    ■■■ ProyVentas_BE ■

ProyVentas_GUI/Dependencias/
  ■■■ Proyectos
    ■■■ ProyVentas_BE ■
    ■■■ ProyVentas_BL ■
```

(NO debe estar ProyVentas_AD0)

/ 7. CONFIGURACIÓN App.config

El archivo App.config almacena la cadena de conexión que la capa ADO leerá en tiempo de ejecución.

PASO 1: Crear el archivo App.config en ProyVentas_GUI

- Clic derecho sobre **ProyVentas_GUI**.
- **Agregar** → **Nuevo elemento...**

■ **NOTA DE VERSIÓN VS:** Pueden aparecer **dos cuadros distintos** según tu versión:

VS 2022 / vista clásica: aparece una ventana grande con plantillas. Busca *configuración* y selecciona **Archivo de configuración de aplicaciones**.

VS 2026 / vista nueva: aparece un cuadro pequeño con un input que dice `FileName.cs`. En ese caso, borra el contenido y escribe directamente **App.config** (con esa mayúscula y esa extensión) y clic en **Agregar**. VS detectará la extensión y lo creará bien.

Si prefieres la vista clásica, haz clic en el enlace **“Mostrar todas las plantillas”** abajo del cuadro pequeño.

PASO 2: Pegar la cadena de conexión

El archivo se abrirá automáticamente. Reemplaza **todo** su contenido por:

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <connectionStrings>
    <add name="Ventas"
      connectionString="Server=.\\SQLEXPRESS;Database=VentasLeon;Integrated Security=true
;TrustServerCertificate=true;"
      providerName="Microsoft.Data.SqlClient" />
    </connectionStrings>
  </configuration>
```

■ **CUIDADO:** El `Server=` debe coincidir con el que usaste en Practica04. Si funcionó con `.\\SQLEXPRESS`, déjalo así. Si usabas `localhost` o `(local)`, cámbialo aquí también.

Guarda con **Ctrl+S**.

/ 8. CAPA DE ENTIDADES (BE)

Las entidades son **clases POCO**: solo propiedades, sin lógica. Representan las tablas y son los objetos que viajan entre capas.

PASO 1: Crear ProductoBE.cs

- Clic derecho sobre ProyVentas_BE → **Agregar** → **Clase...**
- Nombre: **ProductoBE.cs** → **Agregar**.
- Borra TODO el contenido y pega el código siguiente:

ProductoBE.cs — código completo

```
using System;

namespace ProyVentas_BE
{
    /// <summary>
    /// Entidad que representa un registro de la tabla Tb_Producto.
    /// Solo contiene propiedades (datos), sin lógica.
    /// Viaja entre todas las capas (ADO ↔ BL ↔ GUI).
    /// </summary>
    public class ProductoBE
    {
        public string    Cod_pro    { get; set; }
        public string    Des_pro    { get; set; }
        public decimal   Pre_pro    { get; set; }
        public int       Stk_act    { get; set; }
        public int       Stk_min    { get; set; }
        public int       Id_UM      { get; set; }
        public string    Des_UM     { get; set; }
        public int       Id_Cat     { get; set; }
        public string    Des_Cat    { get; set; }
        public int       Importado  { get; set; }
        public DateTime  Fec_Registro { get; set; }
        public string    Usu_Registro { get; set; }
        public DateTime  Fec_Ult_Mod { get; set; }
        public string    Usu_Ult_Mod { get; set; }
        public int       Est_pro    { get; set; }
    }
}
```

Ctrl+S para guardar.

PASO 2: Crear CategoriaBE.cs

```
namespace ProyVentas_BE
{
    public class CategoriaBE
    {
        public int    Id_Cat { get; set; }
        public string Des_Cat { get; set; }
    }
}
```

PASO 3: Crear UnidadMedidaBE.cs

```
namespace ProyVentas_BE
{
    public class UnidadMedidaBE
    {
        public int    Id_UM { get; set; }
        public string Des_UM { get; set; }
    }
}
```

■ **CHECKPOINT: Primer punto de compilación.** Pulsa **Ctrl+Shift+B** y verifica que la ventana de Salida muestre “**4 correctos, 0 erróneos**”. Si hay errores, no continúes — revísalos primero.

/ 9. CAPA DE ACCESO A DATOS (ADO)

Aquí va todo el código que habla con SQL Server: SqlConnection, SqlCommand, SqlDataReader, SqlDataAdapter. Esta capa **no tiene interfaz gráfica** y **no tiene reglas de negocio**: solo ejecuta los procedimientos almacenados.

PASO 1: Crear ConexionADO.cs

Esta clase auxiliar lee la cadena de conexión del App.config de la capa GUI.

```
using System.Configuration;

namespace ProyVentas_ADO
{
    public class ConexionADO
    {
        public string GetCnx()
        {
            string strCnx =
                ConfigurationManager.ConnectionStrings["Ventas"].ConnectionString;

            if (string.IsNullOrEmpty(strCnx))
                return string.Empty;

            return strCnx;
        }
    }
}
```


PASO 2: Crear ProductoADO.cs

Esta es la clase más grande del proyecto: contiene los 5 métodos del CRUD. Cada método invoca a un procedimiento almacenado distinto. **Crea la clase vacía y pega TODO el contenido siguiente** (revisa que no se trunque al copiar):

```
using System;
using System.Data;
using Microsoft.Data.SqlClient;
using ProyVentas_BE;

namespace ProyVentas_ADO
{
    public class ProductoADO
    {
        ConexionADO MiConexion = new ConexionADO();

        // =====
        // 1) LISTAR – Devuelve todos los productos como DataTable
        // =====
        public DataTable ListarProducto()
        {
            using (SqlConnection cnx = new SqlConnection(MiConexion.GetCnx()))
            {
                try
                {
                    SqlCommand cmd = new SqlCommand();
                    cmd.Connection = cnx;
                    cmd.CommandType = CommandType.StoredProcedure;
                    cmd.CommandText = "usp_ListarProducto";

                    SqlDataAdapter ada = new SqlDataAdapter(cmd);
                    DataTable dt = new DataTable();
                    ada.Fill(dt);
                    return dt;
                }
                catch (SqlException ex)
                {
                    throw new Exception("Error al listar productos: " + ex.Message);
                }
            }
        }
    }
}
```

```
// =====
// 2) CONSULTAR – Devuelve UN producto por su código
// =====
public ProductoBE ConsultarProducto(string strCodigo)
{
    ProductoBE obj = null;

    using (SqlConnection cnx = new SqlConnection(MiConexion.GetCnx()))
    {
        try
        {
            SqlCommand cmd = new SqlCommand();
            cmd.Connection = cnx;
            cmd.CommandType = CommandType.StoredProcedure;
            cmd.CommandText = "usp_ConsultarProducto";
            cmd.Parameters.AddWithValue("@vcod", strCodigo);

            cnx.Open();
            SqlDataReader dtr = cmd.ExecuteReader();

            if (dtr.HasRows)
            {
                dtr.Read();
                obj = new ProductoBE
                {
                    Cod_pro = dtr["Cod_pro"].ToString().Trim(),
                    Des_pro = dtr["Des_pro"].ToString(),
                    Pre_pro = Convert.ToDecimal(dtr["Pre_pro"]),
                    Stk_act = Convert.ToInt32(dtr["Stk_act"]),
                    Stk_min = Convert.ToInt32(dtr["Stk_min"]),
                    Id_UM = Convert.ToInt32(dtr["Id_UM"]),
                    Des_UM = dtr["Des_UM"].ToString(),
                    Id_Cat = Convert.ToInt32(dtr["Id_Cat"]),
                    Des_Cat = dtr["Des_Cat"].ToString(),
                    Importado = Convert.ToInt32(dtr["Importado"]),
                    Est_pro = Convert.ToInt32(dtr["Est_pro"])
                };
            }
            dtr.Close();
            return obj;
        }
        catch (SqlException ex)
        {
            throw new Exception("Error al consultar: " + ex.Message);
        }
    }
}
```

```

// =====
// 3) INSERTAR – Recibe una entidad ProductoBE
// =====
public bool InsertarProducto(ProductoBE obj)
{
    using (SqlConnection cnx = new SqlConnection(MiConexion.GetCnx()))
    {
        try
        {
            SqlCommand cmd = new SqlCommand();
            cmd.Connection = cnx;
            cmd.CommandType = CommandType.StoredProcedure;
            cmd.CommandText = "usp_InsertarProducto";

            cmd.Parameters.AddWithValue("@vdes", obj.Des_pro);
            cmd.Parameters.AddWithValue("@vpre", obj.Pre_pro);
            cmd.Parameters.AddWithValue("@vstka", obj.Stk_act);
            cmd.Parameters.AddWithValue("@vstkm", obj.Stk_min);
            cmd.Parameters.AddWithValue("@vId_UM", obj.Id_UM);
            cmd.Parameters.AddWithValue("@vId_Cat", obj.Id_Cat);
            cmd.Parameters.AddWithValue("@vimp", obj.Importado);
            cmd.Parameters.AddWithValue("@vUsu_Registro", obj.Usu_Registro ?? "admin");

            cmd.Parameters.AddWithValue("@vEst_pro", obj.Est_pro);

            cnx.Open();
            cmd.ExecuteNonQuery();
            return true;
        }
        catch (SqlException ex)
        {
            throw new Exception("Error al insertar: " + ex.Message);
        }
    }
}

```

```

// =====
// 4) ACTUALIZAR – Modifica un producto existente
// =====
public bool ActualizarProducto(ProductoBE obj)
{
    using (SqlConnection cnx = new SqlConnection(MiConexion.GetCnx()))
    {
        try
        {
            SqlCommand cmd = new SqlCommand();
            cmd.Connection = cnx;
            cmd.CommandType = CommandType.StoredProcedure;
            cmd.CommandText = "usp_ActualizarProducto";

            cmd.Parameters.AddWithValue("@vcod", obj.Cod_pro);
            cmd.Parameters.AddWithValue("@vdes", obj.Des_pro);
            cmd.Parameters.AddWithValue("@vpre", obj.Pre_pro);
            cmd.Parameters.AddWithValue("@vstka", obj.Stk_act);
            cmd.Parameters.AddWithValue("@vstkm", obj.Stk_min);
            cmd.Parameters.AddWithValue("@vId_UM", obj.Id_UM);
            cmd.Parameters.AddWithValue("@vId_Cat", obj.Id_Cat);
            cmd.Parameters.AddWithValue("@vimp", obj.Importado);
            cmd.Parameters.AddWithValue("@vUsu_ult_mod", obj.Usu_Ult_Mod ?? "admin");

            cmd.Parameters.AddWithValue("@vEst_pro", obj.Est_pro);

            cnx.Open();
            cmd.ExecuteNonQuery();
            return true;
        }
        catch (SqlException ex)
        {
            throw new Exception("Error al actualizar: " + ex.Message);
        }
    }
}

// =====
// 5) ELIMINAR – Borra un producto por su código
// =====
public bool EliminarProducto(string strCodigo)
{
    using (SqlConnection cnx = new SqlConnection(MiConexion.GetCnx()))
    {
        try
        {
            SqlCommand cmd = new SqlCommand();
            cmd.Connection = cnx;
            cmd.CommandType = CommandType.StoredProcedure;
            cmd.CommandText = "usp_EliminarProducto";
            cmd.Parameters.AddWithValue("@vcod", strCodigo);

            cnx.Open();
            cmd.ExecuteNonQuery();
            return true;
        }
        catch (SqlException ex)
        {
            throw new Exception("Error al eliminar: " + ex.Message);
        }
    }
}
}
}
}

```

PASO 3: Crear CategoriaADO.cs

```
using System;
using System.Data;
using Microsoft.Data.SqlClient;

namespace ProyVentas_ADO
{
    public class CategoriaADO
    {
        ConexionADO MiConexion = new ConexionADO();

        public DataTable ListarCategoria()
        {
            using (SqlConnection cnx = new SqlConnection(MiConexion.GetCnx()))
            {
                try
                {
                    SqlCommand cmd = new SqlCommand();
                    cmd.Connection = cnx;
                    cmd.CommandType = CommandType.StoredProcedure;
                    cmd.CommandText = "usp_ListarCategoria";

                    SqlDataAdapter ada = new SqlDataAdapter(cmd);
                    DataTable dt = new DataTable();
                    ada.Fill(dt);
                    return dt;
                }
                catch (SqlException ex)
                {
                    throw new Exception("Error al listar categorías: " + ex.Message);
                }
            }
        }
    }
}
```

PASO 4: Crear UnidadMedidaADO.cs

```
using System;
using System.Data;
using Microsoft.Data.SqlClient;

namespace ProyVentas_ADO
{
    public class UnidadMedidaADO
    {
        ConexionADO MiConexion = new ConexionADO();

        public DataTable ListarUM()
        {
            using (SqlConnection cnx = new SqlConnection(MiConexion.GetCnx()))
            {
                try
                {
                    SqlCommand cmd = new SqlCommand();
                    cmd.Connection = cnx;
                    cmd.CommandType = CommandType.StoredProcedure;
                    cmd.CommandText = "usp_ListarUM";

                    SqlDataAdapter ada = new SqlDataAdapter(cmd);
                    DataTable dt = new DataTable();
                    ada.Fill(dt);
                    return dt;
                }
            }
        }
    }
}
```

```
        }  
        catch (SQLException ex)  
        {  
            throw new Exception("Error al listar UM: " + ex.Message);  
        }  
    }  
}  
}
```

■ **CHECKPOINT: Segundo punto de compilación. Ctrl+Shift+B.** Debe seguir diciendo “4 correctos, 0 erróneos”.

/ 10. CAPA DE NEGOCIO (BL)

La capa BL es el intermediario: aplica validaciones y delega en la ADO. Si las reglas cambian, modificas aquí sin tocar GUI ni BD.

PASO 1: Crear ProductoBL.cs

```
using System;
using System.Data;
using ProyVentas_ADO;
using ProyVentas_BE;

namespace ProyVentas_BL
{
    public class ProductoBL
    {
        ProductoADO objProductoADO = new ProductoADO();

        public DataTable ListarProducto()
        {
            return objProductoADO.ListarProducto();
        }

        public ProductoBE ConsultarProducto(string strCodigo)
        {
            if (string.IsNullOrEmpty(strCodigo))
                throw new Exception("Debe indicar el código del producto.");

            return objProductoADO.ConsultarProducto(strCodigo.Trim());
        }

        public bool InsertarProducto(ProductoBE obj)
        {
            ValidarProducto(obj);
            return objProductoADO.InsertarProducto(obj);
        }

        public bool ActualizarProducto(ProductoBE obj)
        {
            if (string.IsNullOrEmpty(obj.Cod_pro))
                throw new Exception("El código del producto es obligatorio.");
            ValidarProducto(obj);
            return objProductoADO.ActualizarProducto(obj);
        }

        public bool EliminarProducto(string strCodigo)
        {
            if (string.IsNullOrEmpty(strCodigo))
                throw new Exception("Debe indicar el código a eliminar.");
            return objProductoADO.EliminarProducto(strCodigo.Trim());
        }

        private void ValidarProducto(ProductoBE obj)
        {
            if (string.IsNullOrEmpty(obj.Des_pro))
                throw new Exception("La descripción es obligatoria.");
            if (obj.Des_pro.Length > 50)
                throw new Exception("La descripción no puede tener más de 50 caracteres.");
            if (obj.Pre_pro <= 0)
                throw new Exception("El precio debe ser mayor que cero.");
            if (obj.Stk_act < 0)
                throw new Exception("El stock actual no puede ser negativo.");
            if (obj.Stk_min < 0)
                throw new Exception("El stock mínimo no puede ser negativo.");
        }
    }
}
```

```
        throw new Exception("El stock mínimo no puede ser negativo.");
    if (obj.Id_Cat <= 0)
        throw new Exception("Debe seleccionar una categoría.");
    if (obj.Id_UM <= 0)
        throw new Exception("Debe seleccionar una unidad de medida.");
    }
}
```


PASO 2: Crear CategoriaBL.cs

```
using System.Data;
using ProyVentas_ADO;

namespace ProyVentas_BL
{
    public class CategoriaBL
    {
        CategoriaADO objCategoriaADO = new CategoriaADO();

        public DataTable ListarCategoria()
        {
            return objCategoriaADO.ListarCategoria();
        }
    }
}
```

PASO 3: Crear UnidadMedidaBL.cs

```
using System.Data;
using ProyVentas_ADO;

namespace ProyVentas_BL
{
    public class UnidadMedidaBL
    {
        UnidadMedidaADO objUnidadMedidaADO = new UnidadMedidaADO();

        public DataTable ListarUM()
        {
            return objUnidadMedidaADO.ListarUM();
        }
    }
}
```

■ **CHECKPOINT: Tercer punto de compilación. Ctrl+Shift+B.** Debe seguir diciendo “**4 correctos, 0 erróneos**”. Nota que ahora ya hay más proyectos que se recompilan en cascada porque dependen unos de otros.

/ 11. CAPA DE PRESENTACIÓN (GUI)

Vamos a construir un **único formulario** con dos paneles: a la izquierda la grilla con el listado y los botones; a la derecha un panel de edición con los campos del producto. El mismo panel sirve para Agregar y Editar.

PASO 1: Renombrar Form1.cs a FrmProductos.cs

- En el Explorador de soluciones, busca `Form1.cs` dentro de `ProyVentas_GUI`.
- Clic derecho sobre el archivo → **Cambiar nombre** (o pulsa F2).
- Escribe: **FrmProductos.cs** y pulsa Enter.

■ **NOTA:** Visual Studio preguntará: “¿Desea cambiar el nombre de todas las referencias al elemento `Form1` en este proyecto?”. Responde **Sí**. Esto actualiza automáticamente `Program.cs` también.

PASO 2: Reemplazar el código del Designer

- Expande `FrmProductos.cs` con la flechita ■.
- Doble clic sobre `FrmProductos.Designer.cs`.
- Selecciona TODO con `Ctrl+A`.
- Borra y pega el archivo `FrmProductos.Designer.cs` del paquete.
- `Ctrl+S` para guardar.

■ **CUIDADO:** Después de pegar el Designer probablemente verás **muchos errores rojos**. Es **normal y esperado**. Esos errores son porque le faltan los manejadores de eventos (los métodos `btnAgregar_Click`, etc.) que están en el otro archivo. Desaparecerán al pegar el siguiente paso. **NO compiles todavía**.

PASO 3: Reemplazar el código del Form

- Doble clic sobre `FrmProductos` (sin Designer).
- Selecciona TODO con `Ctrl+A`.
- Borra y pega el archivo `FrmProductos.cs` del paquete.
- `Ctrl+S`.

■ **NOTA:** El código de `FrmProductos.cs` es largo (~280 líneas). Contiene la lógica completa del formulario: carga, listado, filtro, agregar, editar, eliminar, validaciones y helpers de panel.

PASO 4: Verificar Program.cs

Abre `Program.cs`. Si renombraste `Form1` correctamente en el paso 1, ya debería decir:

```
Application.Run(new FrmProductos());
```

Si por alguna razón sigue diciendo `new Form1()`, cámbialo manualmente a `new FrmProductos()` y guarda.

■ **CHECKPOINT: Compilación final.** `Ctrl+Shift+B`. Debe decir “**4 correctos, 0 erróneos**”. Si hay errores, revisa el capítulo 13.

/ 12. EJECUCIÓN Y PRUEBAS

PASO 1: Ejecutar la aplicación

- Verifica que ProyVentas_GUI esté en negrita.
- Pulsa **F5** o el botón **Iniciar**.
- Debe abrirse el formulario.

■ CHECKPOINT: Al iniciar:

- La grilla muestra los **5 productos de muestra**.
- El contador dice **“Total registros: 5”**.
- El panel derecho está bloqueado (gris).
- Los combos de Categoría y Unidad de Medida están cargados.

Casos de prueba mínimos

#	Prueba	Resultado esperado
1	AGREGAR → llenar todos los campos → GRABAR	Aparece mensaje de éxito. Nuevo producto P006 en la grilla.
2	AGREGAR → dejar descripción vacía → GRABAR	Mensaje: “La descripción es obligatoria.” (validación BL).
3	AGREGAR → poner precio 0 → GRABAR	Mensaje: “El precio debe ser mayor que cero.” (validación BL).
4	Seleccionar fila → EDITAR	Panel derecho se llena con los datos del producto.
5	Editar la descripción → GRABAR	El cambio se refleja en la grilla.
6	Seleccionar fila → ELIMINAR → confirmar	Pide confirmación; al confirmar, desaparece de la grilla.
7	Escribir texto en el filtro	La grilla se filtra en tiempo real (sin volver a la BD).

/ 13. ERRORES COMUNES Y SOLUCIONES

Este capítulo es nuevo en esta edición revisada. Recopila los problemas más frecuentes que aparecen al hacer esta práctica.

13.1 — Error MSB3026: "No se pudo copiar..."

Síntoma:

```
warning MSB3026: No se pudo copiar "...ProyVentas_GUI.exe".
El archivo se ha bloqueado por: "ProyVentas_GUI (10312)"
```

Causa: Hay un proceso de tu aplicación ejecutándose en segundo plano.

Solución:

- Menú **Depurar** → **Detener depuración** (Shift+F5).
- Si persiste: **Ctrl+Shift+Esc** para abrir Administrador de tareas.
- Busca ProyVentas_GUI.exe en la lista de procesos.
- Clic derecho → **Finalizar tarea**.
- Vuelve a VS y haz **Compilar** → **Limpiar solución**, luego compila de nuevo.

13.2 — "1 correcto, 1 erróneo" sin saber dónde

La ventana de Salida solo dice cuántos proyectos fallaron, pero no muestra los errores claramente. Para verlos:

- Menú **Ver** → **Lista de errores** (Ctrl+\, E).
- Aparece una tabla con columnas: Severidad, Código, Descripción, Proyecto, Línea.
- Doble clic sobre un error → te lleva directo a la línea problemática.

■ **RECUERDA:** Diferencia entre los números de la salida:

Correctos: proyectos que compilaron sin errores en esta compilación.

Actualizados: proyectos que no tenían cambios desde la última compilación.

Erróneos: proyectos con errores de compilación.

13.3 — "No se reconoce el nombre Microsoft.Data.SqlClient"

Causa: Falta el paquete NuGet en ProyVentas_ADO.

Solución: Vuelve al capítulo 5 e instala los paquetes en ProyVentas_ADO.

13.4 — "No se reconoce el nombre ProductoBE" en ADO o BL

Causa: Falta la referencia entre proyectos.

Solución: Revisa el capítulo 6 y verifica las referencias.

13.5 — Cuadro pequeño "FileName.cs" en lugar de plantillas

Es la nueva UI de VS 2026. Para el App.config: borra FileName.cs, escribe App.config directamente y dale Agregar. VS detecta la extensión y crea el archivo correcto.

13.6 — Error al ejecutar: "No se puede conectar al servidor"

Causa: El Server= del App.config no coincide con tu SQL Server.

Solución:

- Abre SSMS y mira en la barra de título a qué servidor estás conectado.
- Copia ese mismo nombre en el App.config.
- Valores comunes: .\SQLEXPRESS, localhost, (local).

13.7 — Error al ejecutar: "Cannot open database VentasLeon"

Causa: No ejecutaste el script SQL del capítulo 3, o falló.

Solución: Vuelve a SSMS, ejecuta el script y verifica con EXEC usp_ListarProducto;.

13.8 — La grilla se ve vacía aunque hay productos

Causa probable: El App.config está conectando a la BD equivocada.

Solución:

- Verifica que Database=VentasLeon en el App.config.
- Verifica con SSMS que la BD VentasLeon existe y tiene productos.
- Si el App.config tiene la cadena correcta pero sigue vacía, ejecuta una consulta directa: `SELECT * FROM VentasLeon.dbo.Tb_Producto;`

/ 14. CONCLUSIONES

- El patrón **N Capas** permite separar responsabilidades: cada cambio se hace en la capa correspondiente sin afectar al resto.
- La **capa de entidades (BE)** es el “idioma común” que hablan todas las capas.
- La **capa de acceso a datos (ADO)** es la única que conoce SQL Server. Cambiar de motor de base de datos no afecta a las demás capas.
- La **capa de negocio (BL)** es donde viven las validaciones. La GUI solo se preocupa de la interfaz.
- Comparado con la **Práctica 04** (monolítico), este enfoque escribe más código al inicio pero es mucho más fácil de mantener y extender.
- El patrón de **compilar después de cada capa** ahorra horas de depuración.

Checklist final del estudiante

- Base de datos VentasLeon creada con tablas, vista y 7 SPs.
- Solución Practica05_Capas con 4 proyectos.
- Paquetes NuGet instalados en ProyVentas_ADO.
- Referencias entre proyectos correctas (GUI→BL→ADO→BE).
- App.config con la cadena "Ventas" en ProyVentas_GUI.
- Clases BE: ProductoBE, CategoriaBE, UnidadMedidaBE.
- Clases ADO: ConexionADO, ProductoADO, CategoriaADO, UnidadMedidaADO.
- Clases BL: ProductoBL, CategoriaBL, UnidadMedidaBL.
- Formulario FrmProductos con el CRUD funcional.
- Las 7 pruebas del capítulo 12 pasan correctamente.

Fin del manual — Práctica 05: Arquitectura en N Capas (Edición Revisada).